Buổi 4

Sắp xếp bằng phương pháp chọn và phương pháp chèn

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Sắp xếp chọn trực tiếp (Selection Sort)

1.1 Selection Sort – tư tưởng, thuật toán, cài đặt
1.2 Selection Sort – minh họa và phân tích

2. Sắp xếp vun đống (HeapSort)

2.1 Cấu trúc Heap
2.2 Hiệu chỉnh heap
2.3 HeapSort – thuật toán và minh họa
2.4 HeapSort – phân tích
2.5 Minh họa HeapSort bằng hình cây

3. Sắp xếp chèn

3.1 Chèn trực tiếp
3.2 Insertion Sort – minh họa và phân tích
3.3 Chèn nhị phân
3.4 Shell Sort – chèn với độ dài bước giảm dần
3.5 Insertion Sort trên mảng cấu trúc và tính ổn định

4. Tổng kết và thực hành

4.1 So sánh các thuật toán chọn và chèn
4.2 Bài thực hành buổi 4
4.3 Câu hỏi và bài tập

1.1 Selection Sort – tư tưởng, thuật toán, cài đặt

- **Tư tưởng:** ở lượt thứ i, **chọn** phần tử **nhỏ nhất** trong đoạn chưa sắp M[i..N−1] rồi **hoán vị** nó với M[i]. Sau N − 1 lượt, dãy có thứ tự. Mỗi lượt chỉ **1 hoán vị**.

```
B1: K = 1                    // vị trí đầu đoạn chưa sắp
B2: IF (K ≥ N) Thực hiện Bkt
B3: PosMin = K               // tìm phần tử nhỏ nhất
B4: Pos = K + 1
B5: IF (Pos > N) Thực hiện B8
B6: IF (M[Pos] < M[PosMin]) PosMin = Pos
B7: Pos++; Lặp lại B5
B8: Swap(M[K], M[PosMin])
B9: K++; Lặp lại B2
Bkt: Kết thúc
```

```
void SelectionSort(T M[], int N)
{  for (int K = 0; K < N - 1; K++)
   {  int PosMin = K;
      for (int Pos = K + 1; Pos < N; Pos++)
         if (M[Pos] < M[PosMin])
            PosMin = Pos;
      if (PosMin != K)
         Swap(M[K], M[PosMin]);
   }
}
```

- **So với Interchange:** cùng so sánh M[i] với các phần tử sau, nhưng chỉ **ghi nhớ vị trí** nhỏ nhất thay vì hoán vị ngay ⇒ tối đa **N − 1** hoán vị.

1.2 Selection Sort – minh họa và phân tích

Ví dụ 4.1 – M = {42, 23, 74, 11, 65, 58, 94, 36}

Lượt K	Đoạn chưa sắp	Min (vị trí)	Hoán vị	Dãy sau lượt
Ban đầu				42 23 74 11 65 58 94 36
0	M[0..7]	11 (3)	42 ↔ 11	**11** 23 74 42 65 58 94 36
1	M[1..7]	23 (1)	không	11 **23** 74 42 65 58 94 36
2	M[2..7]	36 (7)	74 ↔ 36	11 23 **36** 42 65 58 94 74
3	M[3..7]	42 (3)	không	11 23 36 **42** 65 58 94 74
4	M[4..7]	58 (5)	65 ↔ 58	11 23 36 42 **58** 65 94 74
5	M[5..7]	65 (5)	không	11 23 36 42 58 **65** 94 74
6	M[6..7]	74 (7)	94 ↔ 74	11 23 36 42 58 65 **74** 94

Trường hợp	Số phép so sánh	Số hoán vị
Tốt nhất	N(N−1)/2	0
Xấu nhất	N(N−1)/2	N − 1

- Ví dụ trên: 28 phép so sánh, chỉ **4 hoán vị** (Bubble cần 12).

- **Độ phức tạp:** $O(N^2)$ so sánh, nhưng chỉ **O(N)** hoán vị ⇒ tốt khi phép hoán vị **đắt** (phần tử là struct lớn).
- **Ổn định? Không**: (5a, 5b, 3) → lượt 0 đổi 5a với 3 ⇒ (3, 5b, 5a).
- Không tận dụng được dãy đã có thứ tự.

2.1 Cấu trúc Heap

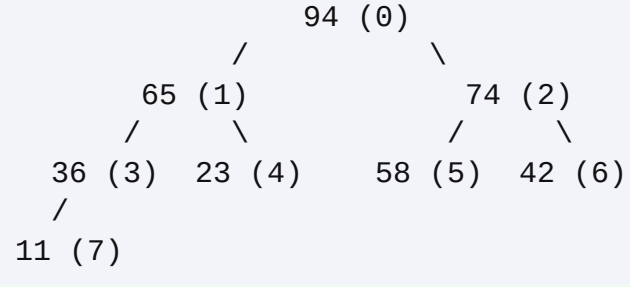
- **Ý tưởng cải tiến Selection:** việc tìm min/max mỗi lượt tốn O(N). Nếu tổ chức dãy thành **Heap**, phần tử lớn nhất luôn ở đầu và việc "hiệu chỉnh" chỉ tốn **O(log N)**.

Định nghĩa Heap (max-heap)

Dãy M[0..N−1] là một **heap** nếu với mọi i: **M[i] ≥ M[2i+1]** và **M[i] ≥ M[2i+2]** (nếu các phần tử đó tồn tại). Các cặp (M[i], M[2i+1]), (M[i], M[2i+2]) gọi là các **liên đới**.

- Có thể xem heap như một **cây nhị phân gần đầy đủ**: nút i có con trái 2i + 1, con phải 2i + 2, cha (i − 1)/2.
- **Tính chất:** M[0] là phần tử **lớn nhất**; mọi nhánh từ gốc xuống lá là dãy giảm.
- Các phần tử từ **N/2** đến N − 1 là **lá** ⇒ tự nó đã là heap.

Ví dụ 4.2 – Dãy 94 65 74 36 23 58 42 11 là heap



- Kiểm tra: 94 ≥ 65, 74; 65 ≥ 36, 23; 74 ≥ 58, 42; 36 ≥ 11 ✓.

2.2 Hiệu chỉnh heap (Shift) và tạo heap ban đầu

- **Shift(M, Left, Right):** giả sử các nhánh con của Left đã là heap, đưa M[Left] **chìm xuống** đúng vị trí: so sánh với con lớn hơn, nếu nhỏ hơn thì hoán vị và tiếp tục xuống dưới.

```
void Shift(T M[], int Left, int Right)
{  int i = Left;
   int j = 2*i + 1;              // con trái
   T X = M[i];
   while (j <= Right)
   {  if (j < Right && M[j] < M[j+1])
         j++;                    // chọn con lớn hơn
      if (M[j] <= X) break;      // đã đúng vị trí
      M[i] = M[j];               // đưa con lên
      i = j;  j = 2*i + 1;
   }
   M[i] = X;
}
```

- **Tạo heap ban đầu:** hiệu chỉnh lần lượt các nút **không phải lá** từ dưới lên: Left = N/2 − 1 xuống 0.

```
void CreateHeap(T M[], int N)
{  for (int Left = N/2 - 1; Left >= 0; Left--)
      Shift(M, Left, N - 1);
}
```

Ví dụ 4.3 – Tạo heap từ M = {42, 23, 74, 11, 65, 58, 94, 36}

Left	Thao tác	Dãy
3	11 < 36 ⇒ đổi	42 23 74 **36** 65 58 94 **11**
2	74 < 94 ⇒ đổi	42 23 **94** 36 65 58 **74** 11
1	23 < 65 ⇒ đổi	42 **65** 94 36 **23** 58 74 11
0	42 < 94 ⇒ đổi; 42 < 74 ⇒ đổi	**94** 65 **74** 36 23 58 **42** 11

2.3 HeapSort – thuật toán và minh họa

- **Thuật toán:**
  - B1: Tạo heap ban đầu từ M[0..N−1].
  - B2: Hoán vị M[0] (lớn nhất) với M[Right] – phần tử cuối đoạn ⇒ phần tử lớn nhất về **đúng vị trí cuối**.
  - B3: Right−−; **Shift(M, 0, Right)** để khôi phục heap.
  - B4: Lặp B2–B3 cho đến khi Right = 0.

```
void HeapSort(T M[], int N)
{  CreateHeap(M, N);
   for (int Right = N - 1; Right > 0; Right--)
   {  Swap(M[0], M[Right]);
      Shift(M, 0, Right - 1);
   }
}
```

Ví dụ 4.4 – Tiếp tục từ heap 94 65 74 36 23 58 42 11

Right	Sau Swap + Shift	Phần đã sắp
7	74 65 58 36 23 11 42	**94**
6	65 42 58 36 23 11	**74 94**
5	58 42 11 36 23	65 74 94
4	42 36 11 23	58 65 74 94
3	36 23 11	42 58 65 74 94
2	23 11	36 42 … 94
1	11	23 36 42 … 94

- Kết quả: **11 23 36 42 58 65 74 94**.

- Chi tiết Right = 7: đổi 94 ↔ 11 ⇒ 11 65 74 36 23 58 42 | 94; Shift: 11 < 74 ⇒ 74 lên; 11 < 58 ⇒ 58 lên ⇒ 74 65 **58** 36 23 **11** 42.

2.4 HeapSort – phân tích

Giai đoạn	Số lần gọi Shift	Chi phí mỗi lần	Tổng
Tạo heap	N/2	$\leq \log_2 N$	O(N) (chặt chẽ hơn N·log N)
Sắp xếp	N − 1	$\leq 2 \cdot \log_2 N$ phép so sánh	O(N log N)

- **Độ phức tạp: O(N log N)** trong **mọi** trường hợp (kể cả xấu nhất) – ưu điểm so với QuickSort.
- **Bộ nhớ phụ:** O(1) (sắp xếp tại chỗ) – ưu điểm so với MergeSort.
- **Ổn định? Không**.
- Thực tế thường chậm hơn QuickSort trung bình (truy cập bộ nhớ "nhảy cóc" 2i + 1).

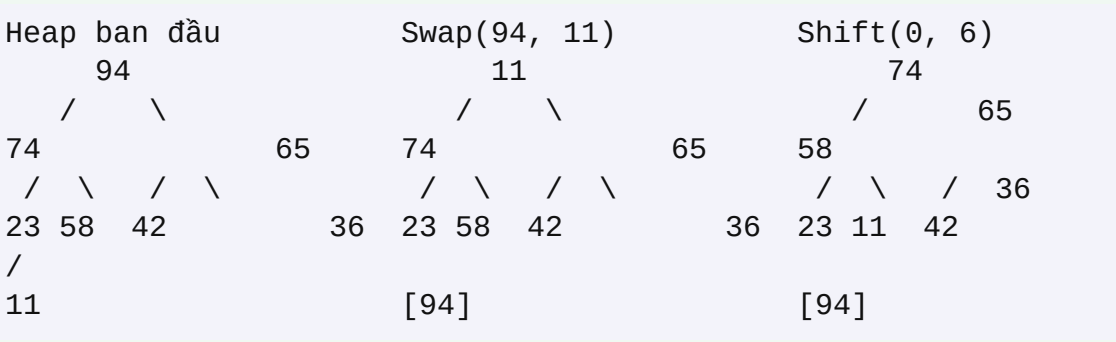
Ví dụ 4.5 – So sánh với Selection Sort, N = 10^6

- Selection: $\approx N^2/2 = 5 \cdot 10^{11}$ so sánh.
- HeapSort: $\approx 2N \cdot \log_2 N \approx 4 \cdot 10^7$ so sánh ⇒ nhanh hơn khoảng **10.000 lần**.
- Cả hai đều "chọn phần tử lớn nhất/nhỏ nhất" – khác nhau ở **CTDL** dùng để chọn (dãy thường vs heap) ⇒ minh họa cho **CTDL + GT = CT**.

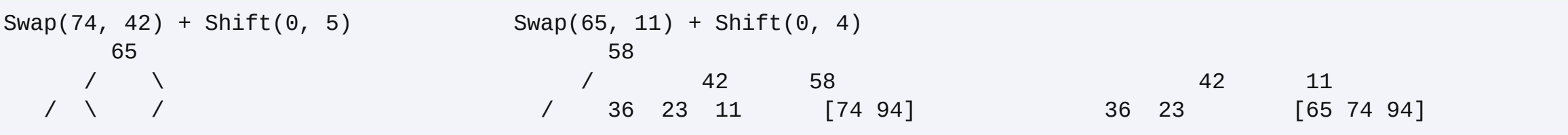
- **Ứng dụng khác của heap:** hàng đợi ưu tiên (lấy phần tử ưu tiên nhất O(log N)), tìm k phần tử lớn nhất.

2.5 Minh họa HeapSort bằng hình cây

Ví dụ 4.4a – Ba bước đầu của giai đoạn sắp xếp (phần tử trong [ ] đã ra khỏi heap)



- **Shift(0, 6)** chi tiết: X = 11 ở gốc; hai con 65, 74 ⇒ chọn **74** (lớn hơn) > 11 ⇒ 74 lên gốc; tại vị trí 2, hai con 58, 42 ⇒ chọn **58** > 11 ⇒ 58 lên; vị trí 5 là lá ⇒ đặt 11.
- Chỉ **2 lần so sánh mỗi mức** × 2 mức = 4 phép so sánh – tỉ lệ với **chiều cao** cây ≈ $\log_2 N$.



- Sau mỗi bước, phần tử lớn nhất còn lại luôn ở **gốc**; phần [ ] tăng dần từ cuối mảng – đó chính là dãy kết quả.

Lỗi thường gặp khi cài đặt

Quên kiểm tra `j < Right` trước khi xét con phải M[j+1] ⇒ truy cập vượt đoạn heap, kéo phần tử **đã sắp** trở lại heap.

3.1 Chèn trực tiếp (Insertion Sort) – tư tưởng và cài đặt

- **Tư tưởng:** như xếp bài trên tay: giả sử đoạn M[0..i−1] **đã có thứ tự**; lấy X = M[i], **dời** các phần tử lớn hơn X sang phải một vị trí, rồi **chèn** X vào chỗ trống. Lặp với i = 1..N−1.

```
B1: K = 2                    // M[1] coi như đã có thứ tự
B2: IF (K > N) Thực hiện Bkt
B3: X = M[K]; Pos = K - 1
B4: IF (Pos < 1 OR M[Pos] ≤ X) Thực hiện B7
B5: M[Pos+1] = M[Pos]        // dời sang phải
B6: Pos--; Lặp lại B4
B7: M[Pos+1] = X             // chèn
B8: K++; Lặp lại B2
Bkt: Kết thúc
```

```
void InsertionSort(T M[], int N)
{  for (int K = 1; K < N; K++)
   {  T X = M[K];
      int Pos = K - 1;
      while (Pos >= 0 && M[Pos] > X)
      {  M[Pos + 1] = M[Pos];
         Pos--;
      }
      M[Pos + 1] = X;
   }
}
```

- Dùng **dời** (1 phép gán) thay cho hoán vị (3 phép gán).
- `M[Pos] > X` (nghiêm ngặt) ⇒ **ổn định**.

3.2 Insertion Sort – minh họa và phân tích

Ví dụ 4.6 – M = {42, 23, 74, 11, 65, 58, 94, 36} (phần gạch dưới: đoạn đã có thứ tự)

K	X	Số phần tử phải dời	Dãy sau khi chèn
1	23	1 (42)	**23** 42 · 74 11 65 58 94 36
2	74	0	23 42 **74** · 11 65 58 94 36
3	11	3 (74, 42, 23)	**11** 23 42 74 · 65 58 94 36
4	65	1 (74)	11 23 42 **65** 74 · 58 94 36
5	58	2 (74, 65)	11 23 42 **58** 65 74 · 94 36
6	94	0	11 23 42 58 65 74 **94** · 36
7	36	5 (94, 74, 65, 58, 42)	11 23 **36** 42 58 65 74 94

- Tổng số lần dời = 1 + 0 + 3 + 1 + 2 + 0 + 5 = **12** = số nghịch thế.

Trường hợp	So sánh	Dời (gán)
Tốt nhất (dãy tăng)	$N - 1$	$2(N - 1)$
Xấu nhất (dãy giảm)	$N(N-1)/2$	$\approx N^2/2$
Trung bình	$\approx N^2/4$	$\approx N^2/4$

- **Độ phức tạp:** $O(N^2)$; nhưng **O(N)** khi dãy gần có thứ tự ⇒ rất tốt cho dữ liệu "gần sắp" và mảng nhỏ (dùng kết hợp trong QuickSort thực tế).
- **Trực tuyến (online):** có thể sắp ngay khi dữ liệu đến từng phần tử.
- Phù hợp với **danh sách liên kết** (chèn không cần dời – buổi 7).

3.3 Chèn nhị phân (Binary Insertion Sort)

- **Cải tiến:** đoạn M[0..K−1] đã có thứ tự ⇒ tìm vị trí chèn bằng **tìm nhị phân** (buổi 2) thay vì so sánh tuần tự ⇒ giảm số phép **so sánh**.

```
void BinaryInsertionSort(T M[], int N)
{  for (int K = 1; K < N; K++)
   {  T X = M[K];
      int First = 0, Last = K - 1;
      while (First <= Last)          // tìm vị trí chèn
      {  int Mid = (First + Last)/2;
         if (X < M[Mid]) Last = Mid - 1;
         else First = Mid + 1;       // X ≥ M[Mid]: sang phải
⇒ ổn định
      }
      for (int Pos = K - 1; Pos >= First; Pos--)
         M[Pos + 1] = M[Pos];        // dời
      M[First] = X;
   }
}
```

Ví dụ 4.7 – Chèn X = 36 vào đoạn 11 23 42 58 65 74 94

First	Last	Mid	M[Mid]	So sánh
0	6	3	58	36 < 58 ⇒ Last = 2
0	2	1	23	36 ≥ 23 ⇒ First = 2
2	2	2	42	36 < 42 ⇒ Last = 1
2	1	–	–	Dừng: chèn tại **2**

- Chỉ **3** phép so sánh (chèn trực tiếp cần 6), nhưng vẫn phải **dời 5** phần tử.

- **Phân tích:** so sánh ≈ $N \cdot \log_2 N$; số lần dời **không đổi** ≈ $N^2/4$ ⇒ vẫn **$O(N^2)$**.

3.4 Shell Sort – chèn với độ dài bước giảm dần

- **Ý tưởng (D. Shell, 1959):** Insertion Sort chậm vì phần tử chỉ dời **1 bước** mỗi lần. Shell chia dãy thành các **dãy con** gồm các phần tử cách nhau **h** vị trí, sắp xếp chèn từng dãy con; giảm dần h (… → 3 → 1). Bước cuối h = 1 là Insertion Sort trên dãy **đã gần có thứ tự** ⇒ rất nhanh.

```
void ShellSort(T M[], int N, int H[], int K)
{  // H[0] > H[1] > … > H[K-1] = 1
   for (int s = 0; s < K; s++)
   {  int h = H[s];
      for (int i = h; i < N; i++)
      {  T X = M[i];
         int j = i - h;
         while (j >= 0 && M[j] > X)
         {  M[j + h] = M[j];
            j -= h;
         }
         M[j + h] = X;
      }
   }
}
```

Ví dụ 4.8 – M = {42, 23, 74, 11, 65, 58, 94, 36}, dãy bước h = 3, 1

- **h = 3:** 3 dãy con theo chỉ số {0, 3, 6}: 42 11 94 → 11 42 94; {1, 4, 7}: 23 65 36 → 23 36 65; {2, 5}: 74 58 → 58 74.
- Sau h = 3: **11 23 58 42 36 74 94 65**.
- **h = 1** (Insertion): chỉ còn **5** lần dời (Insertion Sort trực tiếp cần 12) ⇒ **11 23 36 42 58 65 74 94**.

- **Chọn dãy bước:** Knuth h = 1, 4, 13, 40, … (h = 3h + 1) cho ≈ O(N^1,5); không ổn định.

3.5 Insertion Sort trên mảng cấu trúc và tính ổn định

Ví dụ 4.10 – Sắp danh sách SV theo DTB tăng dần (ổn định)

```
typedef struct {
   char  MaSV[11];
   char  Ten[16];
   float DTB;
} SinhVien;
void InsertionSortSV(SinhVien M[], int N)
{  for (int K = 1; K < N; K++)
   {  SinhVien X = M[K];
      int Pos = K - 1;
      while (Pos >= 0 && M[Pos].DTB > X.DTB)
      {  M[Pos + 1] = M[Pos];
         Pos--;
      }
      M[Pos + 1] = X;
   }
}
```

Trước (đã sắp theo tên)	Sau sắp theo DTB
An 8.0	Bình 7.0
Bình 7.0	Dũng 7.0
Chi 8.0	**An 8.0**
Dũng 7.0	**Chi 8.0**

- An vẫn đứng trước Chi, Bình trước Dũng ⇒ **ổn định**: kết quả là "theo DTB, cùng DTB thì theo tên".
- Nếu đổi `>` thành `>=` ⇒ mất ổn định (Chi sẽ vượt lên trước An).

Mẹo hiệu năng với struct lớn

Mỗi lần dời một SinhVien là sao chép cả struct (≈ 32 byte). Với struct rất lớn, nên sắp xếp **mảng chỉ số** hoặc **mảng con trỏ** tới struct (chỉ di chuyển 4 – 8 byte) – ý tưởng của **sắp xếp theo chỉ mục** (buổi 5).

4.1 So sánh các thuật toán chọn và chèn

Thuật toán	So sánh (xấu nhất)	Hoán vị / dời (xấu nhất)	Tốt nhất	Ổn định	Ghi chú
Selection	$N(N-1)/2$	$N-1$ hoán vị	$O(N^2)$	Không	Ít hoán vị nhất
HeapSort	$\approx 2N \cdot \log_2 N$	$\approx N \cdot \log_2 N$	O(N log N)	Không	Đảm bảo N log N, O(1) bộ nhớ
Insertion	$N(N-1)/2$	$\approx N^2/2$ dời	**O(N)**	Có	Tốt cho dãy gần sắp
Binary Insertion	$\approx N \cdot \log_2 N$	$\approx N^2/2$ dời	O(N log N)	Có	Giảm so sánh, không giảm dời
Shell (Knuth)	≈ N^1,5	≈ N^1,5	O(N log N)	Không	Đơn giản, nhanh với N vừa

Ví dụ 4.9 – Cùng dãy ví dụ (N = 8, 12 nghịch thế): số phép hoán vị / dời thực tế

Bubble	Interchange	Selection	Insertion
12 hoán vị (36 phép gán)	12 hoán vị	**4 hoán vị** (12 phép gán)	12 lần dời + 7 gán X (≈ 26 phép gán)

- **Kết luận:** chọn thuật toán theo đặc điểm dữ liệu: phần tử lớn (đắt khi di chuyển) ⇒ Selection; dữ liệu gần sắp ⇒ Insertion; cần bảo đảm N log N ⇒ HeapSort.

4.2 Bài thực hành buổi 4

Thực hành – Mô phỏng HeapSort và đối chiếu kết quả với chương trình

- 1) Cài đặt `Shift`, `CreateHeap`, `HeapSort`; thêm hàm `InCay(M, N)` in heap theo từng mức: mức 0: M[0]; mức 1: M[1] M[2]; mức 2: M[3..6]…
- 2) Với dãy **{42, 23, 74, 11, 65, 58, 94, 36}**: **trên giấy** vẽ cây heap sau từng bước CreateHeap và sau mỗi lần Swap + Shift; sau đó chạy chương trình in ra và **đối chiếu**.
- 3) Cài đặt SelectionSort, InsertionSort, BinaryInsertionSort, ShellSort (h = 1, 4, 13, 40…) có biến đếm SoSanh, GanHoanVi.
- 4) Chạy 6 thuật toán (thêm Bubble, Quick từ buổi 3) với N = 10.000 trên 3 loại dữ liệu: ngẫu nhiên, tăng dần, giảm dần ⇒ lập bảng.

```
void InCay(T M[], int N)
{  int BatDau = 0, SoNut = 1;
   while (BatDau < N)
   {  for (int k = BatDau; k < BatDau + SoNut && k < N; k++) printf("%4d", M[k]);
      printf("\n");
      BatDau += SoNut;  SoNut *= 2;
   }
}
```

4.3 Câu hỏi và bài tập

Câu hỏi

- 1) So sánh Selection Sort và Interchange Sort về số phép so sánh và hoán vị.
- 2) Heap là gì? Vì sao phần tử đầu heap là lớn nhất? Vì sao các phần tử từ N/2 trở đi không cần hiệu chỉnh?
- 3) Vì sao Insertion Sort chạy rất nhanh với dãy gần có thứ tự? Chèn nhị phân cải tiến được gì, không cải tiến được gì?
- 4) Ý tưởng của Shell Sort? Vì sao bước cuối phải là h = 1?

Bài tập (có đáp số)

- 1) Mô phỏng Selection, Insertion trên dãy **{15, 8, 22, 3, 9}**. → Selection: 3 8 22 15 9 → 3 8 22 15 9 → 3 8 9 15 22 → 3 8 9 15 22 (**2** hoán vị). Insertion: tổng số lần dời = **6**.
- 2) Tạo heap từ dãy {15, 8, 22, 3, 9}. → Left = 1: 8 < 9 ⇒ 15 9 22 3 8; Left = 0: 15 < 22 ⇒ **22 9 15 3 8**.
- 3) Cài đặt HeapSort sắp xếp **giảm dần** (dùng min-heap).
- 4) Dùng heap tìm **10 SV có điểm cao nhất** trong 100.000 SV mà không sắp xếp toàn bộ; đánh giá độ phức tạp.